

Knowvia 知径 macOS Demo 开发说明书

0. 项目基本信息

产品名称

Knowvia 知径

产品副标题

AI 个人知识库与终身学习空间

Slogan

让知识成为路径。

英文 Slogan

Where knowledge becomes your path.

产品定位

Knowvia 知径是一款面向终身学习者的 macOS 本地优先型 AI 个人知识库工具。它帮助用户长期管理课程资料、论文 PDF、笔记、知识卡片和写作素材，将分散的学习资料转化为可持续生长、可追溯、可迁移的个人知识路径。

本阶段目标是先开发一个可运行、可演示、可继续扩展的 macOS Demo，而不是完整商业产品。

1. Demo 开发目标

1.1 核心闭环

Demo 需要完成以下最小可用闭环：

导入 PDF / 文档

- 在资料库中查看
- 打开 PDF 阅读
- 选中文本 / 记录摘录 / 手动创建知识卡片
- 使用 AI 对文档或选中文本进行结构化速读
- 将 AI 摘要或摘录保存为知识卡片
- 在写作工作台调用知识卡片
- 导出 Markdown 草稿

1.2 Demo 要体现的产品价值

1. 不是普通文件夹，而是长期个人知识库；

2. 不是普通 AI 聊天，而是围绕阅读、证据和写作的 AI 学习助手；
 3. 不是代写工具，而是帮助学习者整理资料、追踪证据、形成知识路径的学习支架；
 4. 不是单纯学习打卡，而是面向终身学习的知识资产沉淀工具。
-

2. 第一版开发边界

2.1 本阶段必须做

1. macOS SwiftUI App 基础框架；
2. 三栏式主界面；
3. 本地资料库；
4. 文件导入；
5. PDF 阅读；
6. PDF 文本提取；
7. 文档元数据管理；
8. 知识卡片；
9. AI API Key 设置；
10. OpenAI-compatible API 调用；
11. AI 速读模板；
12. 写作工作台；
13. Markdown 导出；
14. 本地数据保存。

2.2 本阶段不做

第一版 Demo 不做：

1. 登录注册；
 2. 云同步；
 3. iOS 端；
 4. Web 端；
 5. 学习者社区；
 6. 多人协作；
 7. 完整 Zotero 双向同步；
 8. Word 插件；
 9. StoreKit 付费；
 10. 内置 AI 额度；
 11. 复杂知识图谱算法；
 12. OCR；
 13. 上传用户文档到开发者服务器；
 14. 用户行为数据收集；
 15. 一键生成完整论文。
-

3. 技术选型

3.1 平台

macOS 原生应用。

3.2 开发语言与框架

- Swift
- SwiftUI
- PDFKit
- SwiftData 或 Core Data
- Foundation
- Security / Keychain Services
- AppKit interoperability where necessary

3.3 推荐架构

采用 MVVM 架构：

```
Knowvia/  
  App/  
    KnowviaApp.swift  
  
  Models/  
    DocumentItem.swift  
    KnowledgeCard.swift  
    KnowledgeNode.swift  
    KnowledgeRelation.swift  
    WritingProject.swift  
    AIMessage.swift  
    AppSettings.swift  
  
  Views/  
    MainView.swift  
    SidebarView.swift  
    DashboardView.swift  
    LibraryView.swift  
    ReaderView.swift  
    KnowledgeCardsView.swift  
    GraphPlaceholderView.swift  
    WritingView.swift  
    SettingsView.swift  
  
  ViewModels/  
    LibraryViewModel.swift  
    ReaderViewModel.swift  
    CardViewModel.swift  
    AIViewModel.swift  
    WritingViewModel.swift  
    SettingsViewModel.swift  
  
  Services/  
    FileImportService.swift  
    PDFTextExtractionService.swift  
    AIService.swift  
    KeychainService.swift
```

```
MarkdownExportService.swift
LocalPersistenceService.swift
```

Components/

```
DocumentRow.swift
KnowledgeCardView.swift
AIMessageBubble.swift
EvidenceCardView.swift
PrimaryButton.swift
SecondaryButton.swift
TagPill.swift
StatCard.swift
LearningPathStepView.swift
```

Utilities/

```
Color+Hex.swift
DateFormatter+Extensions.swift
PromptTemplates.swift
AppTheme.swift
```

4. 最终 VI 与界面风格

4.1 最终视觉策略

Knowvia 知径最终 VI 采用：

主视觉：Concept A - Path-based Academic Minimalism
辅助情绪：Concept D - Warm Premium Lifelong Learning

即：

- Concept A 作为 App 主界面、Logo、知识库、PDF 阅读、AI 面板、知识卡片的主要风格；
- Concept D 作为 Dashboard、学习路径、成长报告、终身学习愿景相关模块的辅助风格。

4.2 设计原则

主界面应体现：

- 冷静；
- 专业；
- 学术；
- 高级；
- 低干扰；
- 长时间阅读友好；
- Mac 原生工作台感。

成长与学习路径模块可适度体现：

- 温暖；
- 陪伴；
- 长期主义；
- 个人成长；
- 终身学习感。

4.3 不要做成的风格

不要做成：

1. 普通聊天软件；
2. 网盘工具；
3. 纯打卡学习 App；
4. 儿童学习 App；
5. 卡通化教育 App；
6. 过度炫酷的 AI 工具；
7. 五颜六色的知识图谱玩具。

5. 色彩系统

5.1 主色：Concept A

主界面主要使用以下颜色：

```
// Brand / Academic
let deepIndigo = Color(hex: "#1B1E3F")
let knowledgeBlue = Color(hex: "#243B6B")
let slateBlue = Color(hex: "#2F3A5F")

// AI
let softViolet = Color(hex: "#7B6BFF")
let mistPurple = Color(hex: "#A78BFA")
let paleLavender = Color(hex: "#EDE9FE")

// Path / Evidence
let pathTeal = Color(hex: "#21C7C2")
let softTeal = Color(hex: "#8FE3D7")
let paleMint = Color(hex: "#E6FAF7")

// Background
let warmWhite = Color(hex: "#FAFAFB")
let pageBackground = Color(hex: "#F7F8FA")
let cardBackground = Color(hex: "#FFFFFF")
let coolGray = Color(hex: "#E6E8EF")

// Text
let primaryText = Color(hex: "#111827")
```

```
let secondaryText = Color(hex: "#6B7280")
let tertiaryText = Color(hex: "#9CA3AF")
```

5.2 辅助暖色：Concept D

用于 Dashboard、Learning Path、成长模块：

```
// Warm Lifelong Learning Accent
let warmIvory = Color(hex: "#F7F4EF")
let softPlum = Color(hex: "#6B4A6F")
let amberAccent = Color(hex: "#EAA23B")
let seaGlassTeal = Color(hex: "#4DA6A1")
let midnightBlue = Color(hex: "#0F1B2E")
```

5.3 深色模式

```
let darkBackground = Color(hex: "#101218")
let darkCard = Color(hex: "#171A22")
let darkSidebar = Color(hex: "#202432")
let darkBorder = Color(hex: "#303545")
let darkPrimaryText = Color(hex: "#F5F7FA")
let darkSecondaryText = Color(hex: "#A0A7B5")
```

5.4 颜色使用规则

App 主界面

使用 Concept A：

- Deep Indigo：主品牌、标题、选中状态；
- Soft Violet：AI 功能、AI 标签、智能推荐；
- Path Teal：证据链、路径连接、进度；
- Warm White / Cool Gray：背景和分割；
- 不大面积使用 Amber。

Dashboard / 学习路径

融合 Concept D：

- Warm Ivory：背景卡片；
- Amber Accent：成长、连续学习、里程碑；
- Soft Plum：反思类卡片；
- Sea-glass Teal：学习路径进度；
- 仍保持 Deep Indigo 作为文字和主按钮。

PDF 阅读 / 写作区

必须克制：

- 白 / 浅灰 / 深蓝为主；
 - AI 只用少量紫色；
 - 证据链只用少量青绿色；
 - 不使用大面积暖色。
-

6. Logo 与图形语言

6.1 主 Logo

使用 Concept A 的 K/V 路径节点方案。

Logo 结构：

- 抽象 K 或 V；
- 一条路径线；
- 3 个知识节点；
- 渐变从 Deep Indigo / Soft Violet 到 Path Teal；
- 表达 Know + Via，即“知识成为路径”。

6.2 主 App 图标

建议：

深靛蓝圆角矩形背景
+ K/V 路径节点图形
+ 紫色中间节点
+ 青绿色终点节点

6.3 辅助图形

Concept D 的枝芽 / 生长 / 路径图形只作为辅助元素，用于：

- Dashboard 空状态；
- 学习路径；
- 成长报告；
- 官网宣传；
- 终身学习愿景页。

不要用于主 App 图标，避免产品变成习惯打卡或成长类 App。

7. 字体与排版

7.1 字体

使用系统字体：

- 中文：PingFang SC
- 英文：SF Pro

不引入外部字体文件。

7.2 字号建议

Large Title: 28–32
Page Title: 22–26
Section Title: 16–18
Body: 14–16
Caption: 12–13
Tag: 11–12

7.3 排版原则

1. 留白充足；
2. 分割线克制；
3. 标题清晰；
4. 阅读区文字不要太小；
5. AI 输出与用户笔记要视觉区分；
6. 写作区最大宽度控制在 720–820 px。

8. 主界面结构

8.1 三栏布局

采用 macOS 三栏工作台：

左侧 Sidebar：资料库与功能导航
中间 Workspace：阅读 / 卡片 / 图谱 / 写作
右侧 Inspector：AI 助手 / 笔记 / 证据链

推荐使用 `NavigationSplitView`。

8.2 左侧 Sidebar

包含：

1. 首页
2. 最近阅读

3. 资料库
4. 论文库
5. 课程资料
6. 知识卡片
7. 知识图谱
8. 写作项目
9. 学习路径
10. 设置

Demo 可先实现：

- 首页
- 资料库
- 知识卡片
- 写作项目
- 设置

其他页面显示 Coming Soon。

8.3 中间 Workspace

根据导航切换：

- Dashboard
- Library
- Reader
- Knowledge Cards
- Writing
- Graph Placeholder
- Settings

8.4 右侧 Inspector

包含 Tabs：

1. AI 助手
2. 当前笔记
3. 相关卡片
4. 证据链

Demo 优先实现：

- AI 助手；
- 当前文档摘要；
- 相关知识卡片列表；
- 保存 AI 输出为卡片。

9. 页面级 PRD

9.1 Dashboard 首页

视觉参考

Dashboard 可以融合 Concept D 的温暖感，但仍要保持 Concept A 的整体专业气质。

功能目标

展示用户当前学习状态和快速入口。

页面内容

1. 顶部欢迎语：

欢迎来到 Knowvia 知径。
把分散的学习资料，整理成你的长期知识路径。

1. 统计卡片：

2. 文档数量；

3. 知识卡片数量；

4. 写作项目数量；

5. 本周阅读数量。

6. 快捷操作：

7. 导入文档；

8. 新建知识卡片；

9. 新建写作项目；

10. 设置 AI Key。

11. 最近阅读：

展示最近打开的 5 个文档。

1. 最近卡片：

展示最近创建的 5 张知识卡片。

1. 学习路径小组件：

显示一个温和的纵向路径：

开始
基础认知
刻意实践
融会贯通
分享输出

Dashboard 配色

- 背景：Warm White 或 Page Background;
- 卡片：White / Warm Ivory;
- 主按钮：Deep Indigo;
- 路径进度：Path Teal;
- 成长强调：Amber Accent;
- AI 提示：Soft Violet。

9.2 资料库 Library

视觉参考

Concept A。

功能目标

管理用户导入的学习资料。

支持文件类型

Demo 支持：

- PDF
- TXT
- Markdown

功能要求

1. 拖拽导入文件；
2. 按钮导入文件；
3. 显示文档列表；
4. 文档元数据显示：
5. 标题
6. 文件类型
7. 导入时间
8. 标签
9. 阅读状态
10. 支持搜索文档标题；
11. 点击 PDF 后打开 Reader；

12. 点击 TXT / Markdown 后打开文本预览。

空状态文案

还没有资料。
拖入 PDF、课程文档或笔记，开始构建你的第一条知识路径。

9.3 PDF Reader

视觉参考

Concept A。

功能目标

实现基础 PDF 阅读与文本提取。

功能要求

1. 使用 PDFKit 打开 PDF；
2. 支持翻页；
3. 支持缩放；
4. 支持文本选择；
5. 显示页码；
6. 显示文件名；
7. 提供“提取当前页文本”按钮；
8. 提供“AI 速读当前文档”按钮；
9. 提供“从选中文本创建卡片”功能；
10. PDF 阅读区域保持简洁。

标注策略

Demo 阶段不强制做 PDF 原生高亮写回。

可采用：

选中文本 / 复制文本
→ 点击“创建摘录卡片”
→ 保存 `selectedText + documentId + pageNumber`

Reader 右侧 AI 面板

包含：

1. 当前文档标题；
2. AI 速读按钮；
3. AI 摘要结果；

- 4. 保存为知识卡片；
- 5. 当前文档相关知识卡片。

Reader 配色

- 主阅读区：白 / 浅灰；
- 工具栏：白；
- AI 面板：Pale Lavender；
- 证据卡片：Pale Mint；
- 选中/主按钮：Deep Indigo。

9.4 知识卡片 Knowledge Cards

视觉参考

Concept A。

功能目标

将阅读摘录、笔记和 AI 摘要转化为可复用知识卡片。

功能要求

- 1. 创建知识卡片；
- 2. 编辑知识卡片；
- 3. 删除知识卡片；
- 4. 搜索知识卡片；
- 5. 按标签筛选；
- 6. 绑定来源文档；
- 7. 绑定页码；
- 8. 显示卡片类型；
- 9. 显示创建时间；
- 10. 支持将 AI 摘要保存为卡片。

卡片类型

concept	概念
quote	摘录
summary	摘要
method	方法
argument	观点
evidence	证据
question	问题
reflection	反思

卡片颜色规则

- concept: Deep Indigo / 浅蓝底；
- quote: Cool Gray / 白底；

- summary: Soft Violet / Pale Lavender;
- evidence: Path Teal / Pale Mint;
- reflection: Soft Plum / Warm Ivory;
- skill: Amber Accent / Pale Amber。

9.5 AI 助手 AI Assistant

视觉参考

Concept A。

功能目标

提供 BYOK 模式下的 AI 阅读与知识整理能力。

BYOK 策略

用户必须在设置中输入自己的 OpenAI-compatible API Key。Demo 不提供内置模型额度。

功能要求

1. 设置 API Provider Name;
2. 设置 API Endpoint;
3. 设置 API Key;
4. 设置 Model Name;
5. API Key 保存至 Keychain;
6. 测试连接;
7. AI 调用失败时显示错误;
8. 支持对当前文档文本进行结构化摘要;
9. 支持对选中文本进行概念解释;
10. 支持将 AI 输出保存为知识卡片。

默认 API Endpoint

https://api.openai.com/v1/chat/completions

默认 Model

gpt-4o-mini

用户可以自定义 endpoint 和 model，以支持 OpenAI-compatible 服务。

AI 输出提示

AI 输出卡片必须显示：

AI 输出仅作为阅读和写作支架。请结合原文核验，不要直接作为最终论文内容使用。

9.6 写作工作台 Writing Workspace

视觉参考

Concept A。

功能目标

帮助用户把知识卡片、摘录和证据组织成写作素材。

功能要求

1. 创建写作项目；
2. 编辑项目标题；
3. 编辑正文 Markdown；
4. 右侧显示可插入知识卡片；
5. 点击卡片后插入到当前正文；
6. 支持插入引用占位；
7. 支持导出 Markdown。

插入卡片格式

> [知识卡片] 卡片标题
> 来源：文档标题，p.页码
> 摘录/摘要内容……

AI 写作限制

Demo 不做“一键生成论文”。

只允许：

- 整理提纲；
- 根据已选卡片生成段落支架；
- 提醒用户核验来源。

9.7 学习路径 Learning Path Placeholder

视觉参考

Concept D。

Demo 阶段目标

学习路径暂时作为展示/占位模块，不实现复杂算法。

显示内容

展示一个固定路径组件：

开始：设定学习目标
基础认知：理解核心概念
刻意实践：建立可持续行动
融会贯通：形成自己的知识体系
分享输出：让知识产生影响

视觉要求

- 使用 Warm Ivory 背景；
- 使用 Path Teal 作为路径线；
- 当前步骤使用 Deep Indigo；
- 完成步骤可用 Amber Accent 或 Teal 小标记；
- 风格温暖但不花哨。

9.8 设置 Settings

功能目标

管理 AI Key、模型配置和隐私选项。

设置项

1. API Provider Name；
2. API Endpoint；
3. API Key；
4. Model Name；
5. Test Connection；
6. Clear API Key；
7. 本地数据路径；
8. 隐私说明。

隐私说明文案

Knowvia 采用本地优先设计。你导入的文档、笔记和写作草稿默认保存在本机。AI 功能需要将你选择的文本发送到你配置的模型服务商。API Key 将保存在 macOS Keychain 中。

10. 数据模型

10.1 DocumentItem

```
@Model
final class DocumentItem {
    var id: UUID
    var title: String
    var filePath: String
    var fileType: String
    var importedAt: Date
    var updatedAt: Date
    var tags: [String]
    var readingStatus: String
    var lastOpenedAt: Date?
    var pageCount: Int?
    var extractedText: String?
    var summary: String?

    init(
        title: String,
        filePath: String,
        fileType: String,
        importedAt: Date = Date(),
        updatedAt: Date = Date(),
        tags: [String] = [],
        readingStatus: String = "unread"
    ) {
        self.id = UUID()
        self.title = title
        self.filePath = filePath
        self.fileType = fileType
        self.importedAt = importedAt
        self.updatedAt = updatedAt
        self.tags = tags
        self.readingStatus = readingStatus
    }
}
```

10.2 KnowledgeCard

```
@Model
final class KnowledgeCard {
    var id: UUID
    var title: String
    var content: String
    var summary: String?
    var cardType: String
}
```

```

var tags: [String]
var sourceDocumentId: UUID?
var sourceDocumentTitle: String?
var pageNumber: Int?
var createdAt: Date
var updatedAt: Date
var createdBy: String

init(
    title: String,
    content: String,
    cardType: String = "note",
    tags: [String] = [],
    sourceDocumentId: UUID? = nil,
    sourceDocumentTitle: String? = nil,
    pageNumber: Int? = nil,
    createdBy: String = "user"
){
    self.id = UUID()
    self.title = title
    self.content = content
    self.cardType = cardType
    self.tags = tags
    self.sourceDocumentId = sourceDocumentId
    self.sourceDocumentTitle = sourceDocumentTitle
    self.pageNumber = pageNumber
    self.createdAt = Date()
    self.updatedAt = Date()
    self.createdBy = createdBy
}
}

```

10.3 KnowledgeNode

```

@Model
final class KnowledgeNode {
    var id: UUID
    var name: String
    var nodeType: String
    var nodeDescription: String
    var tags: [String]
    var createdAt: Date
    var updatedAt: Date

    init(
        name: String,
        nodeType: String = "concept",
        nodeDescription: String = "",
        tags: [String] = []
    ){

```

```

        self.id = UUID()
        self.name = name
        self.nodeType = nodeType
        self.nodeDescription = nodeDescription
        self.tags = tags
        self.createdAt = Date()
        self.updatedAt = Date()
    }
}

```

10.4 WritingProject

```

@Model
final class WritingProject {
    var id: UUID
    var title: String
    var projectType: String
    var draftContent: String
    var linkedCardIds: [UUID]
    var createdAt: Date
    var updatedAt: Date

    init(
        title: String,
        projectType: String = "essay",
        draftContent: String = ""
    ) {
        self.id = UUID()
        self.title = title
        self.projectType = projectType
        self.draftContent = draftContent
        self.linkedCardIds = []
        self.createdAt = Date()
        self.updatedAt = Date()
    }
}

```

10.5 AIMessage

```

struct AIMessage: Identifiable, Codable {
    var id: UUID = UUID()
    var role: String
    var content: String
    var createdAt: Date = Date()
}

```

10.6 AppSettings

```
struct AppSettings: Codable {  
    var providerName: String  
    var apiEndpoint: String  
    var modelName: String  
}
```

API Key 不允许存入 AppSettings、UserDefaults 或 SwiftData，必须通过 KeychainService 保存。

11. AI Prompt 模板

11.1 文档速读 Prompt

你是一个学术阅读助手。请对以下文档内容进行结构化速读，但不要编造文档中没有的信息。

请按以下格式输出：

1. 核心主题
2. 关键概念
3. 主要观点
4. 重要证据
5. 可转化为知识卡片的内容
6. 可能用于写作的引用点
7. 用户需要继续追问的问题

要求：

- 使用中文输出；
- 保持简洁；
- 如果原文信息不足，请明确说明“原文未提供”；
- 不要替用户直接写完整论文；
- 尽量保留原文术语。

文档内容如下：

{{document_text}}

11.2 概念解释 Prompt

请解释以下概念，并帮助学习者理解它在当前文档中的作用。

输出格式：

1. 概念定义
2. 在当前文档中的含义
3. 与其他概念的关系
4. 可以如何记成知识卡片

5. 可能的学习提醒

文本：

{{selected_text}}

11.3 写作支架 Prompt

你是一个学术写作支架助手。请根据用户选择的知识卡片，帮助用户整理成一段写作思路，而不是直接代写最终论文。

要求：

- 先给出段落结构；
- 再给出可展开的观点链；
- 标注每个观点对应的卡片来源；
- 提醒用户核验原文；
- 不编造引用。

知识卡片如下：

{{selected_cards}}

12. AIService 设计

12.1 请求格式

使用 OpenAI-compatible chat completions。

12.2 Service 接口

```
final class AIService {  
  func sendChatCompletion(  
    endpoint: String,  
    apiKey: String,  
    model: String,  
    messages: [AIMessage]  
  ) async throws -> String  
}
```

12.3 错误处理

至少处理：

- API Key 为空；
- endpoint 为空；
- 网络失败；
- 401 / 403 认证错误；
- 429 速率限制；

- 500 服务端错误；
- 响应解析失败；
- 内容过长。

12.4 内容长度控制

Demo 阶段采用简单截断：

如果提取文本超过 12000 字符，只取前 12000 字符，并提示用户 Demo 阶段仅处理部分内容。

后续版本再做文档分块、向量检索和本地 RAG。

13. KeychainService 设计

13.1 目标

安全保存用户 API Key。

13.2 功能接口

```
final class KeychainService {  
    func saveAPIKey(_ key: String) throws  
    func loadAPIKey() throws -> String?  
    func deleteAPIKey() throws  
}
```

13.3 要求

1. 不要把 API Key 打印到日志；
2. 不要把 API Key 存入 UserDefaults；
3. 不要把 API Key 存入 SwiftData；
4. 不要把 API Key 写进 crash log；
5. 设置页输入框必须使用 secure field；
6. 用户可一键删除 API Key。

14. 文件导入与本地存储

14.1 文件导入方式

支持：

1. NSOpenPanel 选择文件；
2. 拖拽文件到资料库。

14.2 文件保存策略

Demo 阶段：

- 导入时复制文件到 App 沙盒目录；
- 建议路径：Documents/KnowviaLibrary；
- 数据库中保存本地副本路径；
- 内部 ID 使用 UUID；
- UI 显示原文件名。

14.3 支持文件

- PDF
 - TXT
 - Markdown
-

15. PDF 文本提取

15.1 提取策略

使用 PDFKit 读取 PDFDocument，并遍历 page.string。

15.2 Service 接口

```
final class PDFTextExtractionService {  
    func extractText(from url: URL) -> String  
    func extractText(from url: URL, pageIndex: Int) -> String  
}
```

15.3 失败提示

如果 PDF 无法提取文本，提示：

该 PDF 可能是扫描版，当前 Demo 暂不支持 OCR。

16. Markdown 导出

16.1 导出对象

支持导出：

- 写作项目正文；
- 插入的知识卡片；
- 来源信息。

16.2 导出格式

项目标题

正文内容……

相关知识卡片

卡片标题

类型：concept

来源：文档标题，p.12

卡片内容……

16.3 导出方式

使用 NSSavePanel 选择保存路径。

17. 隐私与数据原则

Demo 阶段必须遵守：

1. 用户文档默认保存在本地；
2. 不上传用户文档到开发者服务器；
3. 不收集用户行为数据；
4. 不收集用户 API Key；
5. 不收集 PDF 正文；
6. AI 调用只发送用户主动选择的文本或当前文档提取文本；
7. 用户需要清楚知道 AI 请求会发送给其配置的模型服务商；
8. API Key 保存在 Keychain；
9. Demo 不做科研数据采集。

18. Demo 文案

首页欢迎语

欢迎来到 Knowvia 知径。

把分散的学习资料，整理成你的长期知识路径。

空资料库

还没有资料。
拖入 PDF、课程文档或笔记，开始构建你的第一条知识路径。

空知识卡片

还没有知识卡片。
你可以从 PDF 摘录、AI 摘要或自己的笔记中创建第一张卡片。

AI Key 未配置

AI 助手尚未启用。
请在设置中添加你的 OpenAI-compatible API Key。Knowvia 不会保存或上传你的 Key，它将存储在本机 Keychain 中。

AI 摘要提示

AI 输出仅作为阅读和写作支架。请结合原文核验，不要直接作为最终论文内容使用。

19. 开发优先级

P0：必须完成

1. SwiftUI App 初始化；
2. 三栏主界面；
3. 资料库页面；
4. 文件导入；
5. PDF Reader；
6. PDF 文本提取；
7. 知识卡片创建 / 编辑 / 删除；
8. 设置页；
9. Keychain 保存 API Key；
10. AIService 基础调用；
11. AI 文档速读；
12. 写作项目；
13. Markdown 导出；
14. 基础 VI 风格落地。

P1：重要但可后置

1. 标签筛选；
2. 文档搜索；
3. 卡片搜索；
4. 最近阅读；

5. 首页统计;
6. 卡片插入写作项目;
7. 简单知识图谱占位页;
8. 深色模式优化;
9. 学习路径占位组件。

P2: Demo 不要求

1. OCR;
 2. 云同步;
 3. App Store IAP;
 4. 复杂知识图谱布局;
 5. 社区;
 6. 多模型管理;
 7. Zotero 同步;
 8. Word 导出;
 9. 学习分析;
 10. 数据看板。
-

20. Codex 开发执行顺序

请按顺序开发，不要一次性堆满所有功能。

Step 1: 项目骨架与主题系统

- 创建 macOS SwiftUI App;
- 建立项目文件夹结构;
- 添加 Color+Hex;
- 添加 AppTheme;
- 使用 Concept A 主配色;
- 预留 Concept D 暖色 token;
- 创建 MainView;
- 使用 NavigationSplitView 搭建三栏布局;
- 创建 SidebarView;
- 创建 DashboardView 占位。

Step 2: 资料库

- 创建 DocumentItem 模型;
- 创建 LibraryView;
- 实现文件导入;
- 将文件复制到 App Library;
- 保存 DocumentItem;
- 显示文档列表;
- 做空状态;
- 做基础搜索。

Step 3: PDF Reader

- 用 PDFKit 包装 PDFView;

- 支持打开选中文档；
- 显示 PDF；
- 支持缩放 / 翻页；
- 实现文本提取；
- 添加 “AI 速读” 按钮；
- 添加 “创建摘录卡片” 入口。

Step 4: 知识卡片

- 创建 KnowledgeCard 模型；
- 创建 KnowledgeCardsView；
- 实现增删改查；
- 实现从选中文本 / AI 摘要创建卡片；
- 卡片样式使用 Concept A；
- reflection / growth 卡片可使用 Concept D 暖色。

Step 5: 设置与 Keychain

- 创建 SettingsView；
- 创建 KeychainService；
- 保存、读取、删除 API Key；
- 配置 endpoint 和 model；
- 显示隐私说明。

Step 6: AIService

- 创建 AIService；
- 实现 OpenAI-compatible 请求；
- 添加 PromptTemplates；
- 在 Reader 右侧 AI 面板显示结果；
- 支持保存 AI 输出为知识卡片；
- 处理常见错误。

Step 7: 写作工作台

- 创建 WritingProject 模型；
- 创建 WritingView；
- 支持新建写作项目；
- 支持编辑 Markdown 文本；
- 支持插入知识卡片；
- 支持导出 Markdown。

Step 8: Dashboard 与学习路径占位

- Dashboard 加入统计卡片；
- 加入最近文档；
- 加入最近知识卡片；
- 加入学习路径小组件；
- 学习路径使用 Concept D 的温暖路径视觉；
- 不做复杂算法。

Step 9: 统一 UI 与体验

- 统一按钮；
 - 统一卡片；
 - 统一标签；
 - 统一空状态；
 - 统一错误提示；
 - 优化窗口尺寸；
 - 确保主界面像 Concept A；
 - 确保 Dashboard 和 Learning Path 有 Concept D 的温暖感，但不能变成打卡 App。
-

21. 验收标准

Demo 完成后应满足：

1. App 可以成功运行；
 2. 主界面采用三栏布局；
 3. 基础视觉符合 Concept A；
 4. Dashboard 有 Concept D 的温暖成长感；
 5. 可以导入 PDF；
 6. 可以在资料库中看到 PDF；
 7. 可以打开 PDF 阅读；
 8. 可以提取 PDF 文本；
 9. 可以配置 API Key；
 10. API Key 保存在 Keychain；
 11. 可以调用 AI 生成文档速读摘要；
 12. 可以把 AI 摘要保存为知识卡片；
 13. 可以手动创建知识卡片；
 14. 可以新建写作项目；
 15. 可以把卡片插入写作项目；
 16. 可以导出 Markdown；
 17. API Key 不出现在日志、数据库、UserDefaults 中；
 18. 用户文档默认本地保存；
 19. Demo 不上传用户文档；
 20. AI 写作功能不提供一键生成论文。
-

22. 需要避免的实现方式

开发时请不要：

1. 不要做登录注册；
2. 不要做云同步；
3. 不要上传文件到任何服务器；
4. 不要写死开发者自己的 API Key；
5. 不要把 API Key 存到 UserDefaults；
6. 不要生成复杂后端；
7. 不要引入大型第三方依赖；
8. 不要做 Electron；

9. 不要用 WebView 伪装原生 App;
10. 不要把 AI 聊天作为唯一主界面;
11. 不要做“一键生成论文”功能;
12. 不要实现付费系统;
13. 不要实现社区功能;
14. 不要过早优化复杂知识图谱算法;
15. 不要把整个产品做成学习打卡 App。

23. Demo 成功后的下一阶段

Demo 完成后再考虑：

1. PDF 原生高亮写回;
2. BibTeX / RIS 导入;
3. 文献元数据自动识别;
4. 知识图谱节点关系;
5. 文档分块与本地 RAG;
6. 本地 embeddings;
7. Ollama 本地模型支持;
8. 学习时间线;
9. 技能图谱;
10. App Store 上架;
11. StoreKit 买断版;
12. 用户同意后的匿名学习分析;
13. 学习路径共享;
14. 学习者社区。

给 Codex 的一句话任务

请基于以上说明，使用 SwiftUI 开发一个 macOS 本地 Demo，名称为 Knowvia 知径。第一版只实现本地资料库、PDF 阅读、知识卡片、BYOK AI 速读、写作工作台、Dashboard 学习路径占位和 Markdown 导出，不做账号、云同步、社区和付费功能。主视觉参考 Concept A 的路径化学术极简风，Dashboard 和学习路径模块吸收 Concept D 的温暖终身学习感。代码结构要清晰，优先保证可运行、可演示、可继续扩展。